

# Guia Rápido: Primeiro Projeto com LangChain

## 1. Chamada Simples a um LLM

```
# 01_basico.py
import os, httpx
from dotenv import load_dotenv
load_dotenv()

API_KEY = os.getenv("OPENROUTER_API_KEY")

def perguntar(prompt, modelo="google/gemini-2.5-flash-lite"):
    resp = httpx.post(
        "https://openrouter.ai/api/v1/chat/completions",
        headers={
            "Authorization": f"Bearer {API_KEY}",
            "Content-Type": "application/json",
        },
        json={
            "model": modelo,
            "messages": [{"role": "user", "content": prompt}],
        },
        timeout=30,
    )
    return resp.json()["choices"][0]["message"]["content"]

# Teste
print(perguntar("Explique o que é LangChain em 1 frase."))
print(perguntar("Qual a capital do Brasil?", "openai/gpt-4o"))
```

## 2. Memória de Conversa

```
# 02_memoria.py
import os, httpx
from dotenv import load_dotenv
load_dotenv()

API_KEY = os.getenv("OPENROUTER_API_KEY")
historico = []

def conversar(msg, modelo="google/gemini-2.5-flash-lite"):
    historico.append({"role": "user", "content": msg})
```

```

resp = httpx.post(
    "https://openrouter.ai/api/v1/chat/completions",
    headers={
        "Authorization": f"Bearer {API_KEY}",
        "Content-Type": "application/json",
    },
    json={
        "model": modelo,
        "messages": historico, # envia histórico todo
    },
    timeout=30,
).json()

resposta = resp["choices"][0]["message"]["content"]
historico.append({"role": "assistant", "content": resposta})
return resposta

print(conversar("Meu nome é Rodolfo, sou de Campinas."))
print(conversar("Qual meu nome e de onde eu sou?"))
# O modelo lembra porque o histórico está no array

```

### 3. System Prompt (Personalidade)

```

# 03_persona.py
import os, httpx
from dotenv import load_dotenv
load_dotenv()

API_KEY = os.getenv("OPENROUTER_API_KEY")

SYSTEM = "Você é Jarvis, um assistente de IA estilo Homem de Ferro. Seja conciso e use português."

def jarvis(prompt):
    resp = httpx.post(
        "https://openrouter.ai/api/v1/chat/completions",
        headers={
            "Authorization": f"Bearer {API_KEY}",
            "Content-Type": "application/json",
        },
        json={
            "model": "openai/gpt-4o",
            "messages": [
                {"role": "system", "content": SYSTEM},
                {"role": "user", "content": prompt},
            ],
        },
        timeout=30,
    ).json()
    return resp["choices"][0]["message"]["content"]

```

```
print(jarvis("Qual a temperatura hoje?"))
```

## 4. Tool Calling (Function Calling)

```
# 04_tools.py
import os, httpx, json
from datetime import datetime
from dotenv import load_dotenv
load_dotenv()

API_KEY = os.getenv("OPENROUTER_API_KEY")

TOOLS = [{
    "type": "function",
    "function": {
        "name": "get_time",
        "description": "Retorna data e hora atual",
        "parameters": {"type": "object", "properties": {}},
        "required": []},
    },
]

def executar_ferramenta(nome, args):
    if nome == "get_time":
        return datetime.now().strftime("%H:%M:%S de %d/%m/%Y")
    return "Ferramenta não encontrada"

def perguntar_com_tools(prompt):
    messages = [{"role": "user", "content": prompt}]

    resp = httpx.post(
        "https://openrouter.ai/api/v1/chat/completions",
        headers={
            "Authorization": f"Bearer {API_KEY}",
            "Content-Type": "application/json",
        },
        json={
            "model": "google/gemini-2.5-flash-lite",
            "messages": messages,
            "tools": TOOLS,
            "tool_choice": "auto",
        },
        timeout=30,
    ).json()

    msg = resp["choices"][0]["message"]

    if msg.get("tool_calls"):
        for tc in msg["tool_calls"]:
            nome = tc["function"]["name"]
            args = json.loads(tc["function"]["arguments"])
```

```

        resultado = executar_ferramenta(nome, args)

        messages.append(msg)
        messages.append({
            "role": "tool",
            "tool_call_id": tc["id"],
            "content": resultado,
        })

# Segunda chamada com o resultado da tool
    resp = httpx.post(
        "https://openrouter.ai/api/v1/chat/completions",
        headers={
            "Authorization": f"Bearer {API_KEY}",
            "Content-Type": "application/json",
        },
        json={
            "model": "google/gemini-2.5-flash-lite",
            "messages": messages,
        },
        timeout=30,
    ).json()
    return resp["choices"][0]["message"]["content"]

    return msg.get("content", "")

print(perguntar_com_tools("Que horas são agora?"))

```

## 5. LangChain: Prompt Templates

```

# 05_langchain_prompt.py
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "Você é um {profissao} especialista em {area}."),
    ("user", "{pergunta}"),
])

template = prompt.invoke({
    "profissao": "copywriter",
    "area": "marketing digital",
    "pergunta": "Crie uma headline para um curso de IA",
})

print("System:", template.messages[0].content)
print("User:", template.messages[1].content)
# Envie para o LLM com o código dos exemplos anteriores

```

## 6. LangChain: Chains

```
# 06_chain.py
import os, httpx
from langchain_core.prompts import ChatPromptTemplate
from dotenv import load_dotenv
load_dotenv()

API_KEY = os.getenv("OPENROUTER_API_KEY")
MODEL = "google/gemini-2.5-flash-lite"

prompt = ChatPromptTemplate.from_messages([
    ("system", "Você é um especialista em {topico}. Responda em português."),
    ("user", "{pergunta}"),
])

def llm_call(messages):
    """Função que chama o LLM - adapte para seu provider"""
    resp = httpx.post(
        "https://openrouter.ai/api/v1/chat/completions",
        headers={"Authorization": f"Bearer {API_KEY}", "Content-Type": "application/json"},
        json={"model": MODEL, "messages": messages},
        timeout=30,
    ).json()
    return resp["choices"][0]["message"]["content"]

# Pipeline simples: prompt → LLM
pergunta = "O que é RAG em IA?"
topico = "inteligência artificial generativa"

messages = prompt.invoke({"topico": topico, "pergunta": pergunta})
mensagens_formatadas = [
    {"role": m.type if m.type != "system" else "system",
     "content": m.content}
    for m in messages.messages
]
resposta = llm_call(mensagens_formatadas)
print(resposta)
```

## Modelos Disponíveis (OpenRouter)

| Modelo                | ID                           | Custo  | Uso             |
|-----------------------|------------------------------|--------|-----------------|
| Gemini 2.5 Flash Lite | google/gemini-2.5-flash-lite | Grátis | Tarefas simples |
| GPT-4o Mini           | openai/gpt-4o-mini           | Baixo  | Rápido e bom    |
| GPT-4o                |                              | Médio  | Qualidade       |

| Modelo          | ID                                | Custo  | Uso                   |
|-----------------|-----------------------------------|--------|-----------------------|
|                 | openai/<br>gpt-4o                 |        |                       |
| Claude Sonnet 4 | anthropic/<br>claude-<br>sonnet-4 | Médio  | Melhor<br>qualidade   |
| DeepSeek Chat   | deepseek/<br>deepseek-<br>chat    | Grátis | Alternativa<br>grátis |

**Próximo:** [03-langgraph-intro.md](#)